

Session 22 (AIML) - Assignment Questions

- **Name of Student** : Shruti Dnyaneshwar Darwatkar
- **Domain Name**: AIML Development
- **College Name**: Zeal Polytechnic , Narhe
- **Branch Name**: AIML
- **Github url**: https://github.com/shrutidarwatkar-sketch/Task_ITR.git

Heart Disease Prediction Using Streamlit

CODES:

Q1. Data Loading & Preprocessing

```
# Q1

import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
from google.colab import files
uploaded = files.upload()

df = pd.read_csv("heart.csv")
|
# Separate features and target
X = df.drop("HeartDisease", axis=1)
y = df["HeartDisease"]

# Encode categorical columns
label_encoders = {}

for col in X.select_dtypes(include="object").columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    label_encoders[col] = le

print("Features:")
print(X.head())

print("\nTarget:")
print(y.head())
```

```

... Choose files heart.csv
heart.csv(text/csv) - 35921 bytes, last modified: 09/07/2026 - 100% done
Saving heart.csv to heart.csv
Features:
  Age  Sex  ChestPainType  RestingBP  Cholesterol  FastingBS  RestingECG  \
0   40   1             1         140          289          0          1
1   49   0             2         160          180          0          1
2   37   1             1         130          283          0          2
3   48   0             0         138          214          0          1
4   54   1             2         150          195          0          1

  MaxHR  ExerciseAngina  Oldpeak  ST_Slope
0    172              0      0.0         2
1    156              0      1.0         1
2     98              0      0.0         2
3    108              1      1.5         1
4    122              0      0.0         2

Target:
0     0
1     1
2     0
3     1
4     0
Name: HeartDisease, dtype: int64

```

Q2. Train-Test Split

```

#Q2

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("X_train Shape:", X_train.shape)
print("X_test Shape :", X_test.shape)
print("y_train Shape:", y_train.shape)
print("y_test Shape :", y_test.shape)

... X_train Shape: (734, 11)
X_test Shape : (184, 11)
y_train Shape: (734,)
y_test Shape : (184,)

```

Q3. Building Logistic Regression Model



#Q3

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

print("Logistic Regression Model Trained Successfully!")
```

```
... Logistic Regression Model Trained Successfully!
```

Q4. Making Predictions



#Q4

```
y_pred = model.predict(X_test)

print("First 10 Actual Values:")
print(y_test.values[:10])

print("\nFirst 10 Predicted Values:")
print(y_pred[:10])
```

```
... First 10 Actual Values:
[0 1 1 1 0 1 1 0 1 1]
```

```
First 10 Predicted Values:
[0 0 1 1 0 1 1 0 0 1]
```

Q5. Confusion Matrix

```
#Q5
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix")
print(cm)

TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

print("\nTrue Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
print("True Positive (TP):", TP)
```

```
... Confusion Matrix
[[68  9]
 [20 87]]

True Negative (TN): 68
False Positive (FP): 9
False Negative (FN): 20
True Positive (TP): 87
```

Q6. Model Evaluation Metrics



#Q6

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

```
... Accuracy : 0.842391304347826
Precision: 0.90625
Recall   : 0.8130841121495327
F1 Score : 0.8571428571428571
```

Classification Report				
	precision	recall	f1-score	support
0	0.77	0.88	0.82	77
1	0.91	0.81	0.86	107
accuracy			0.84	184
macro avg	0.84	0.85	0.84	184
weighted avg	0.85	0.84	0.84	184

Q7. Saving the Model

```
import joblib

joblib.dump(model, "heart_model.pkl")
joblib.dump(label_encoders, "encoders.pkl")
joblib.dump(list(X.columns), "columns.pkl")

... ['columns.pkl']
```

Q8. Loading & Testing Saved Model

```
#Q8
import joblib
import pandas as pd

# Load saved files
model = joblib.load("heart_model.pkl")
encoders = joblib.load("encoders.pkl")
columns = joblib.load("columns.pkl")

# Sample patient record
sample = {
    "Age":40,
    "Sex":"M",
    "ChestPainType":"ATA",
    "RestingBP":140,
    "Cholesterol":289,
    "FastingBS":0,
    "RestingECG":"Normal",
    "MaxHR":172,
    "ExerciseAngina":"N",
    "Oldpeak":0.0,
    "ST_Slope":"Up"
}

sample_df = pd.DataFrame([sample])
```

```

sample_df = pd.DataFrame([sample])

# Encode categorical columns
for col in sample_df.columns:
    if col in encoders:
        sample_df[col] = encoders[col].transform(sample_df[col])

prediction = model.predict(sample_df)

if prediction[0] == 1:
    print("Heart Disease: Yes")
else:
    print("Heart Disease: No")

```

... Heart Disease: No

+ Code

+ Text

Q9. Streamlit Input Interface

import streamlit as st

st.title("Heart Disease Prediction")

Input Fields

age = st.number_input("Age")

sex = st.selectbox("Sex", ["M", "F"])

cp = st.selectbox(
 "Chest Pain Type",
 ["ATA", "NAP", "ASY", "TA"]
)

)

bp = st.number_input("Resting Blood Pressure")

chol = st.number_input("Cholesterol")

fbs = st.selectbox(
 "Fasting Blood Sugar",
 [0, 1]
)

ecg = st.selectbox(
 "Resting ECG",
 ["Normal", "ST", "LVH"]
)

hr = st.number_input("Maximum Heart Rate")

angina = st.selectbox(
 "Exercise Angina",
 ["Y", "N"]
)

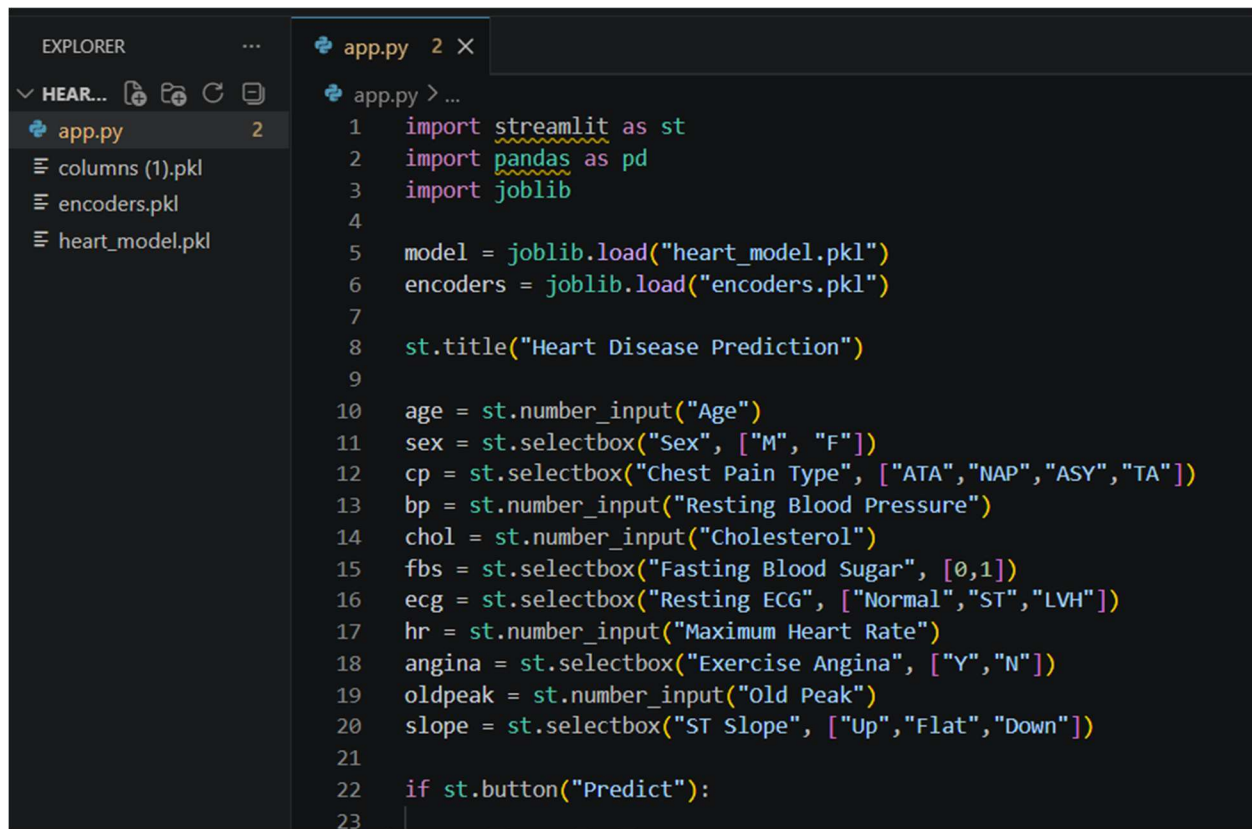

```
oldpeak = st.number_input("Old Peak")
```

```
slope = st.selectbox(  
    "ST Slope",  
    ["Up", "Flat", "Down"]  
)
```

```
# Predict Button
```

```
if st.button("Predict"):  
    st.write("Predict button clicked!")
```

Q10. Complete Streamlit Web App



```
EXPLORER
HEAR...
app.py 2
columns (1).pkl
encoders.pkl
heart_model.pkl

app.py > ...
1 import streamlit as st
2 import pandas as pd
3 import joblib
4
5 model = joblib.load("heart_model.pkl")
6 encoders = joblib.load("encoders.pkl")
7
8 st.title("Heart Disease Prediction")
9
10 age = st.number_input("Age")
11 sex = st.selectbox("Sex", ["M", "F"])
12 cp = st.selectbox("Chest Pain Type", ["ATA", "NAP", "ASY", "TA"])
13 bp = st.number_input("Resting Blood Pressure")
14 chol = st.number_input("Cholesterol")
15 fbs = st.selectbox("Fasting Blood Sugar", [0,1])
16 ecg = st.selectbox("Resting ECG", ["Normal", "ST", "LVH"])
17 hr = st.number_input("Maximum Heart Rate")
18 angina = st.selectbox("Exercise Angina", ["Y", "N"])
19 oldpeak = st.number_input("Old Peak")
20 slope = st.selectbox("ST Slope", ["Up", "Flat", "Down"])
21
22 if st.button("Predict"):
23
```

```
HEARTDISEASEPROJECT
app.py 2
columns (1).pkl
encoders.pkl
heart_model.pkl

app.py > ...
17 oldpeak = st.number_input("Old Peak",)
20 slope = st.selectbox("ST Slope", ["Up", "Flat", "Down"])
21
22 if st.button("Predict"):
23
24     data = pd.DataFrame([
25         "Age": age,
26         "Sex": sex,
27         "ChestPainType": cp,
28         "RestingBP": bp,
29         "Cholesterol": chol,
30         "FastingBS": fbs,
31         "RestingECG": ecg,
32         "MaxHR": hr,
33         "ExerciseAngina": angina,
34         "Oldpeak": oldpeak,
35         "ST_Slope": slope
36     ])
37
38 for col in data.columns:
39     if col in encoders:
40         data[col] = encoders[col].transform(data[col])
41
42 prediction = model.predict(data)
43
44 if prediction[0] == 1:
45     st.error("Heart Disease: Yes")
46 else:
47     st.success("Heart Disease: No")
```

OUTPUTS

1. App.py output

```
PS C:\Users\tanve\Desktop\HeartDiseaseProject> python -m streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.11:8501
```

2. Streamlit app home page

Heart Disease Prediction

Age

0.00 - +

Sex

M ▾

Chest Pain Type

NAP ▾

Resting Blood Pressure

0.00 - +

Cholesterol

0.00 - +

Fasting Blood Sugar

0 ▾

Resting ECG

Normal ▾

Maximum Heart Rate

0.00 - +

Exercise Angina

N ▾

Old Peak

0.00 - +

ST Slope

Up ▾

Predict

Heart Disease: No

Heart Disease Prediction

Age

0.00

- +

Sex

M

▼

Chest Pain Type

ATA

▼

Resting Blood Pressure

0.00

- +

Cholesterol

0.00

- +

Fasting Blood Sugar

0

▼

Resting ECG

Normal

▼

Maximum Heart Rate

0.00

- +

Exercise Angina

Y

▼

Old Peak

0.00

- +

ST Slope

Up

▼

Predict

Heart Disease: Yes

Heart Disease Prediction

Age

0.00

- +

Sex

M

▼

Chest Pain Type

ATA

▼

Resting Blood Pressure

0.00

- +

Cholesterol

16.00

- +

Fasting Blood Sugar

0

▼

Resting ECG

ST

▼

Maximum Heart Rate

0.00

- +

Exercise Angina

Y

▼

Old Peak

0.00

- +

ST Slope

Up

▼

Predict

Heart Disease: No

Heart Disease Prediction

Age

0.00

- +

Sex

M

▼

Chest Pain Type

ATA

▼

Resting Blood Pressure

0.00

- +

Cholesterol

12.00

- +

Fasting Blood Sugar

1

▼

Resting ECG

ST

▼

Maximum Heart Rate

0.00

- +

Exercise Angina

Y

▼

Old Peak

0.00

- +

ST Slope

Up

▼

Predict

Heart Disease: Yes

➤ Conclusion:

- Successfully loaded and preprocessed the **Heart Disease** dataset by encoding categorical features.
- Split the dataset into training and testing sets using `train_test_split()`.
- Built and trained a **Logistic Regression** model for heart disease prediction.
- Evaluated the model using a **Confusion Matrix, Accuracy, Precision, Recall, F1-Score**, and **Classification Report**.
- Saved the trained model and preprocessing objects using **Joblib** for future use.
- Developed a **Streamlit web application** with input fields and a prediction button for user interaction.
- Successfully integrated the trained model into the Streamlit application to predict whether a patient is likely to have heart disease.
- This project demonstrates the complete machine learning workflow, including **data preprocessing, model training, evaluation, model saving/loading, and deployment** in a simple web application.